

PROJECT REPORT

OSCC-MS Subtype Classifier: Machine Learning Classification of the Mesenchymal Molecular Subtype of Head and Neck Squamous Cell Carcinoma from Gene Expression Data

Course: Fundamentals of AI / ML

Flipped Course Evaluation - Project Submission

Domain: Machine Learning Basics - Supervised Learning / Classification

Student Name: _____

Roll No / Student ID: _____

GitHub Repository: <https://github.com/<your-username>/<your-repo-name>>

Date: September 2026

1. Introduction

Head and Neck Squamous Cell Carcinoma (HNSCC), which includes Oral Squamous Cell Carcinoma (OSCC), is one of the more common cancers worldwide and is far from a single uniform disease. Large-scale genomic work by The Cancer Genome Atlas (TCGA) consortium showed that HNSCC tumors actually split into four distinct molecular subtypes based on their gene expression patterns: Basal, Mesenchymal, Atypical, and Classical (Walter et al., 2013; The Cancer Genome Atlas Network, 2015). Of these, the Mesenchymal (MS) subtype stands out clinically -- it's driven by an epithelial-to-mesenchymal transition (EMT) expression signature and is generally associated with a more invasive tumor phenotype.

This project applies a supervised machine learning approach -- specifically a Random Forest classifier -- to the problem of predicting whether a given tumor sample belongs to the Mesenchymal subtype or not, based purely on its gene expression profile. Rather than working with synthetic or simulated data, this project deliberately uses real, publicly available patient data from the TCGA-HNSC cohort, obtained through the UCSC Xena Browser and the original TCGA Nature 2015 publication's supplementary materials.

The motivation for choosing this topic came directly from the course's Machine Learning Basics module -- classification, feature importance, the bias-variance tradeoff, and the curse of dimensionality are all concepts that come up naturally when building a model like this on real biological data, where the number of possible features (genes) vastly exceeds the number of available patient samples.

2. Problem Statement

Identifying a tumor's molecular subtype currently requires either whole-genome expression clustering (computationally heavy and not practical as a quick, per-patient check) or manual review of dozens of marker genes by someone with specialist domain knowledge. This project asks a narrower, directly answerable version of that problem:

Given a patient's expression values for a defined, literature-grounded panel of marker genes, can a machine learning model reliably classify a head and neck tumor as Mesenchymal (MS) versus Other (Basal / Atypical / Classical), while remaining explainable rather than a black box?

2.1 Scope

- Binary classification only -- Mesenchymal (MS) vs Other. Full 4-class subtyping is out of scope for this project (see Future Enhancements).
- Restricted to a curated 35-gene panel grounded in existing HNSCC literature, rather than the full ~20,000-gene expression matrix -- this keeps the model interpretable and avoids the curse of dimensionality on a relatively small (279-patient) real-world cohort.
- Built entirely on real, publicly available TCGA-HNSC data -- no synthetic or simulated data is used anywhere in the final model.
- Delivered as a command-line pipeline (data ingestion, preprocessing, training, evaluation, prediction) rather than a GUI or web app, to keep the project fully scriptable, reproducible, and CLI-executable.
- Does not cover clinical deployment, regulatory validation, or integration with real hospital systems -- this is an academic proof-of-concept, not a certified diagnostic tool.

2.2 Target Users

- Primary (for this submission): the course evaluator, assessing correct application of supervised ML concepts to a real dataset.
- Illustrative real-world users (the scenario this project models): bioinformatics researchers or lab technicians who have gene expression data for a sample and want a fast, explainable first-pass subtype classification to help prioritize follow-up work.

3. Functional Requirements

The system is built around five functional modules, each with a clearly defined input/output boundary, chained together into one command-line workflow (main.py).

ID	Module	Input	Output
FR-1	Data Ingestion (data_module.py)	Raw TCGA expression matrix (.gz) + subtype label file (.xlsx)	Merged, cleaned CSV dataset (patients x genes + label)
FR-2	Preprocessing (preprocessing_module.py)	Merged dataset CSV	Schema-validated data; stratified train/test split
FR-3	Model Training (training_module.py)	Training split (X_train, y_train)	Trained Random Forest model + CV metrics + saved model file
FR-4	Model Evaluation (evaluation_module.py)	Trained model + test split	Accuracy/precision/recall/F1/ROC-AUC, confusion matrix, ROC curve, feature-importance plots
FR-5	Prediction (prediction_module.py)	New patient CSV (35 gene values)	Predicted subtype, class probabilities, top contributing genes

3.1 User Workflow

The intended end-to-end workflow, matching the four CLI subcommands, is:

- 1. `python main.py build-dataset --` merge the two raw source files into the modelling dataset (run once, or whenever raw data changes)
- 2. `python main.py train --` train the Random Forest with 5-fold cross-validation and persist the model
- 3. `python main.py evaluate --` score the held-out test set and generate evaluation plots
- 4. `python main.py predict --input <patient.csv> --` classify a new patient and return an explainable prediction

This satisfies the "at least three functional modules with clear input/output and logical workflow" requirement with five modules rather than the minimum three, and directly implements the course's classification / prediction ML use case.

4. Non-Functional Requirements

Requirement	How it's addressed
Reliability	Every dataset and every prediction input is schema-validated (correct columns, no nulls, numeric types, known labels) before it reaches the model. Malformed input raises a clear, specific error instead of silently producing a wrong prediction.
Maintainability	All hyperparameters, file paths, and the gene panel live in one config.py. Each pipeline stage is an independent module with a single responsibility, so changing one stage (e.g. swapping the model type) does not require touching the others.
Usability	A single, consistent CLI (main.py) drives the whole pipeline with four simple subcommands. Predictions are returned as clean, human-readable JSON.
Logging / Monitoring	Every module logs to both the console and a persistent logs/app.log file (via a shared logger.py), giving an auditable record of what happened on each run -- dataset sizes, CV scores, metrics, errors.
Scalability	The gene panel, model hyperparameters, and even the underlying dataset can be swapped by editing config.py alone. Data ingestion is fully decoupled from training/evaluation/prediction, so a different cohort or a different classifier could be substituted without changing the downstream modules.
Resource Efficiency	Random Forest was deliberately chosen over a deep learning approach given the small sample size (279 patients) -- training completes in well under one second on a standard laptop CPU, with no GPU requirement.

This covers six non-functional requirements, exceeding the minimum of four requested.

5. System Architecture

The system follows a linear, pipeline-style architecture: raw public data sources feed into a data-ingestion stage, which produces a clean dataset consumed by preprocessing, training, and evaluation in sequence. A separately trained model artifact is then reused by the prediction module to score new, unseen patients. A shared configuration and logging layer (config.py / logger.py) is used across every stage, which is what keeps the modules consistent without duplicating constants.

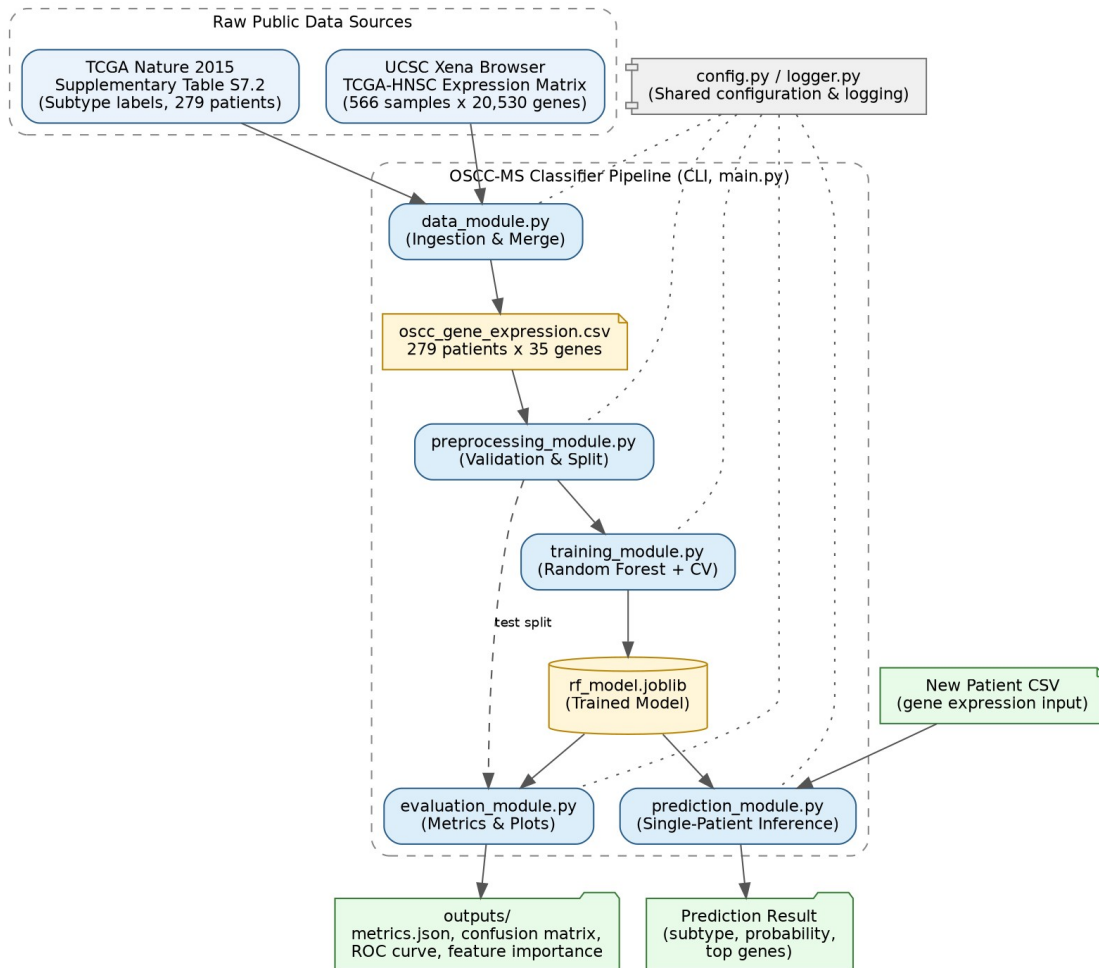


Figure 1: System Architecture -- data flow from raw public sources through to trained model and predictions.

6. Design Diagrams

6.1 Use Case Diagram

The primary actor is the researcher / evaluator running the pipeline. The core use cases map directly onto the five functional modules and the CLI subcommands; "Validate Input Schema" is an included use case shared by both dataset-building and prediction.

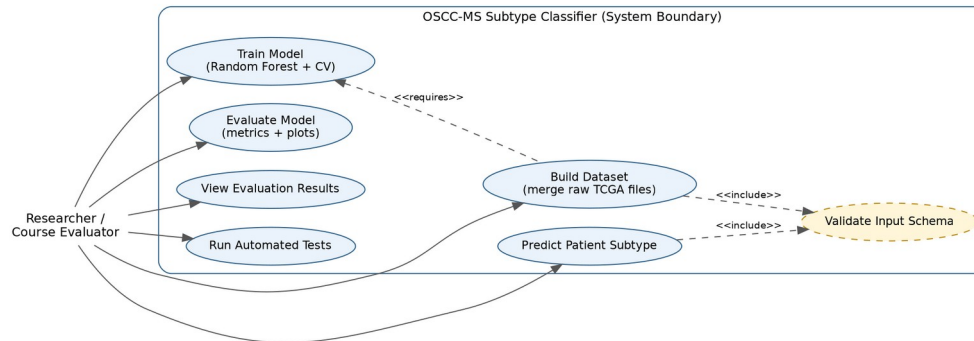


Figure 2: Use Case Diagram.

6.2 Process Flow / Workflow Diagram

The workflow below traces a full run of the pipeline from start to finish across all four CLI stages, including the input-validation branch in the prediction stage.

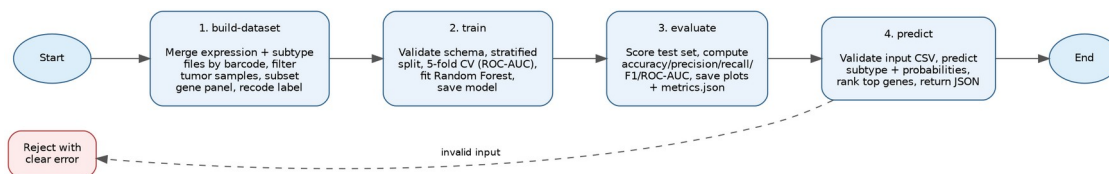


Figure 3: Process Flow / Workflow Diagram.

6.3 Class / Component Diagram

Since this project is implemented as a set of focused, function-based Python modules rather than a deep class hierarchy, the diagram below is presented as a component diagram: each box represents one module, its public functions, and its dependencies on the shared config and logger components.

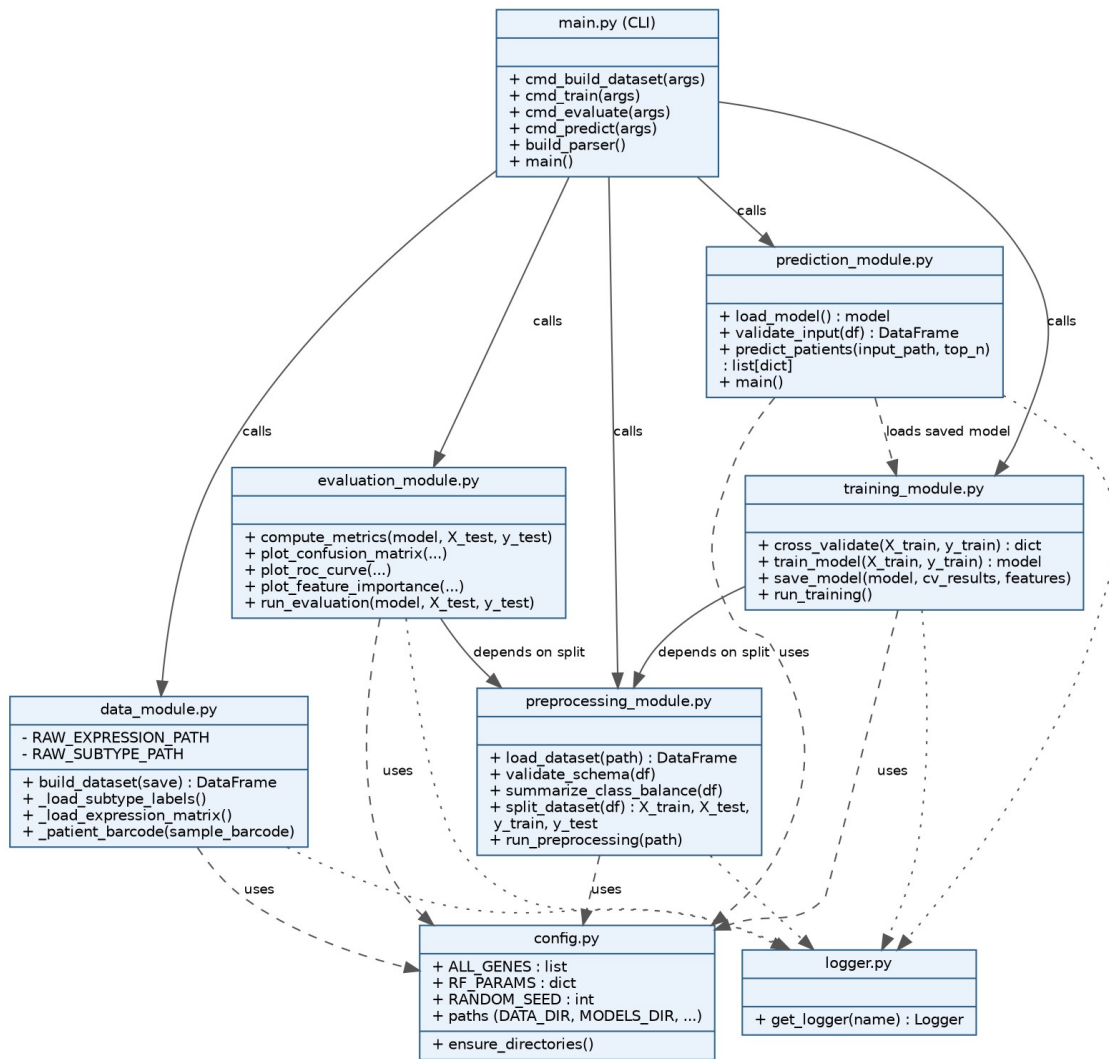


Figure 4: Class / Component Diagram.

6.4 Sequence Diagram

The sequence diagram below traces the most user-facing use case -- predicting a new patient's subtype -- from the CLI command down to the model and back.

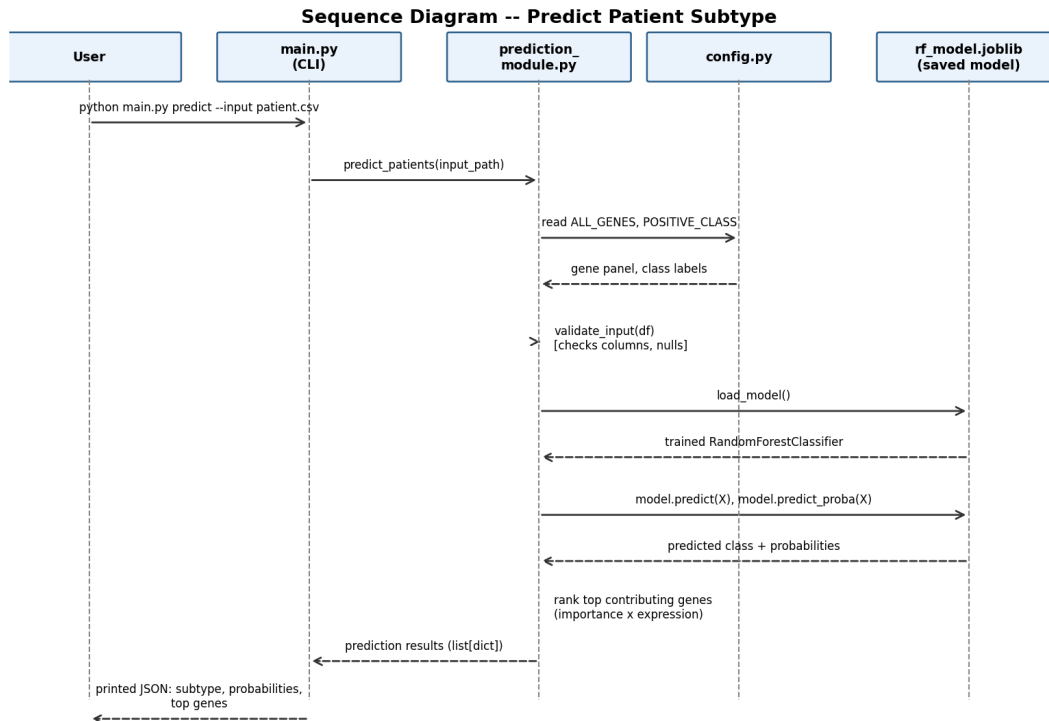


Figure 5: Sequence Diagram -- Predict Patient Subtype.

6.5 Database / Storage Design

This project does not use a relational database, so a formal ER Diagram is not applicable. All persistent state is stored as flat files: the merged dataset and prediction inputs/outputs are CSV/JSON files, and the trained model is serialized with joblib. The schema of the core dataset file (data/oscc_gene_expression.csv) is documented below in place of a formal ER diagram, since it plays the same role a table schema would in a database-backed system.

Column	Type	Description
sample_id	string	TCGA sample barcode, e.g. TCGA-CR-7383-01 (primary key / unique identifier)
<35 gene columns>	float	log2(RSEM+1) normalized expression value for each gene in the literature panel
subtype_full	string	Original 4-class TCGA RNA subtype (Basal / Mesenchymal / Atypical / Classical) -- kept for reference
subtype	string	Binary modelling label derived from subtype_full: "MS" or "Other"

7. Design Decisions & Rationale

7.1 Why Random Forest

Random Forest was chosen over alternatives like logistic regression, SVM, or a neural network for a few concrete reasons. It handles the high-feature, low-sample-count regime of biological expression data reasonably well without heavy tuning; it doesn't require feature scaling; and, critically for this project, it produces feature importances directly, which is what makes the per-prediction "top contributing genes" explanation possible. A neural network would very likely overfit on only 279 samples and would not offer the same built-in interpretability.

7.2 Why a Curated 35-Gene Panel Instead of the Full Transcriptome

The raw expression matrix has over 20,000 genes, but only 279 labelled patients. Feeding all 20,000 genes into a model trained on 279 samples is a textbook case of the curse of dimensionality -- the model would have far more features than observations, making overfitting almost guaranteed and feature importances essentially meaningless. Restricting the model to a 35-gene panel grounded in the EMT/mesenchymal-transition literature keeps the feature-to-sample ratio sane, keeps the model interpretable, and means the "top genes" the model reports are genes a domain expert can actually reason about.

7.3 Why Binary (MS vs Other) Instead of Full 4-Class

The original TCGA labels have four classes, but the smallest class (Classical, 49 patients) would leave very few samples per cross-validation fold if split four ways. Framing the problem as MS vs Other keeps the class sizes large enough (75 vs 204) for the 5-fold stratified cross-validation to be meaningful, while still answering the clinically relevant question -- MS is the subtype of primary clinical interest due to its association with a more invasive phenotype.

7.4 Why Stratified Cross-Validation and Balanced Class Weights

The dataset is imbalanced (75 MS vs 204 Other, roughly 27%/73%). Two things address this directly: stratified splitting (both in the CV folds and the train/test split) ensures every fold preserves that same ratio rather than risking a fold with almost no MS examples, and `class_weight="balanced"` in the Random Forest penalizes misclassifying the minority (MS) class more heavily during training, so the model isn't just rewarded for always predicting the majority class.

7.5 Why a CLI-First Design

The course's project executability requirement is explicit about avoiding GUI-based setups, so the entire project is built as a scriptable command-line pipeline from the start, rather than a CLI wrapped around what was originally meant to be a GUI or notebook. This also makes it trivially reproducible -- every stage is one command, and every command is testable in isolation.

8. Implementation Details

8.1 Technology Stack

- Python 3.10+, argparse for the CLI
- pandas / numpy for data loading and manipulation
- scikit-learn (RandomForestClassifier, StratifiedKFold, cross_val_score, train_test_split, metrics)
- matplotlib / seaborn for the confusion matrix, ROC curve, and feature-importance plots
- openpyxl for reading the raw Excel-format subtype label file
- joblib for model persistence
- pytest for the unit test suite

8.2 Data Ingestion Logic

The two raw source files use different patient identifier formats -- the expression matrix uses sample barcodes like TCGA-CR-7383-01 (where the -01 suffix denotes a primary tumor sample, and -11 denotes adjacent normal tissue), while the subtype label file uses dot-separated patient barcodes like TCGA.BA.4074. The ingestion module normalizes both to a common patient-level barcode, keeps only primary tumor samples, and performs an inner join on that barcode. Of the 566 total samples in the expression matrix, 520 are primary tumor samples, and 279 of those have a matching subtype label -- this became the final modelling cohort.

8.3 Model Configuration

RandomForestClassifier(n_estimators=300, max_depth=None, min_samples_leaf=2, class_weight="balanced", random_state=42). All hyperparameters are centralized in config.py rather than hardcoded inside training_module.py, so they can be changed in one place.

8.4 Error Handling Strategy

Each module defines its own narrow exception type (DataIngestionError, SchemaValidationError, PredictionInputError) rather than letting raw exceptions like KeyError or FileNotFoundError propagate up uninterpreted. main.py catches these at the CLI boundary, logs a clear message, and exits with a non-zero status code -- so a malformed CSV or a missing file produces an understandable error message instead of a stack trace.

9. Screenshots / Results

9.1 Evaluation Metrics

Metric	Value
5-fold CV ROC-AUC (mean)	0.957
Test set accuracy	0.90
Test set precision (MS)	0.88
Test set recall (MS)	0.74
Test set F1-score (MS)	0.80
Test set ROC-AUC	0.964

9.2 Confusion Matrix

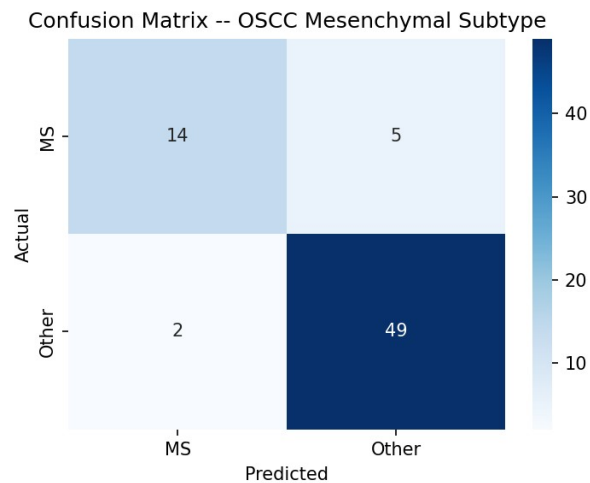


Figure 6: Confusion Matrix on the held-out test set (14/19 MS cases correctly identified).

9.3 ROC Curve

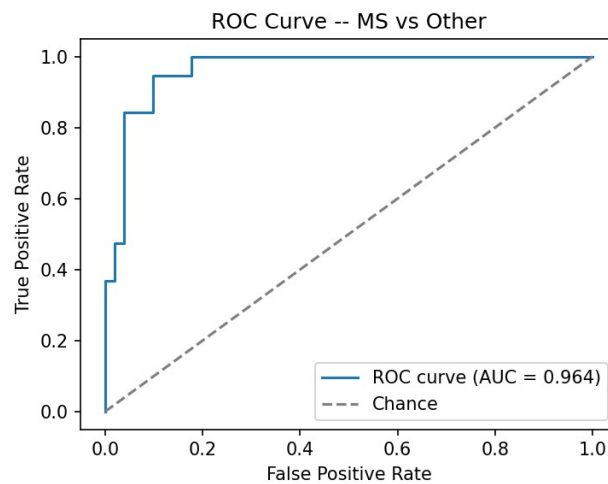


Figure 7: ROC Curve, AUC = 0.964.

9.4 Feature Importance

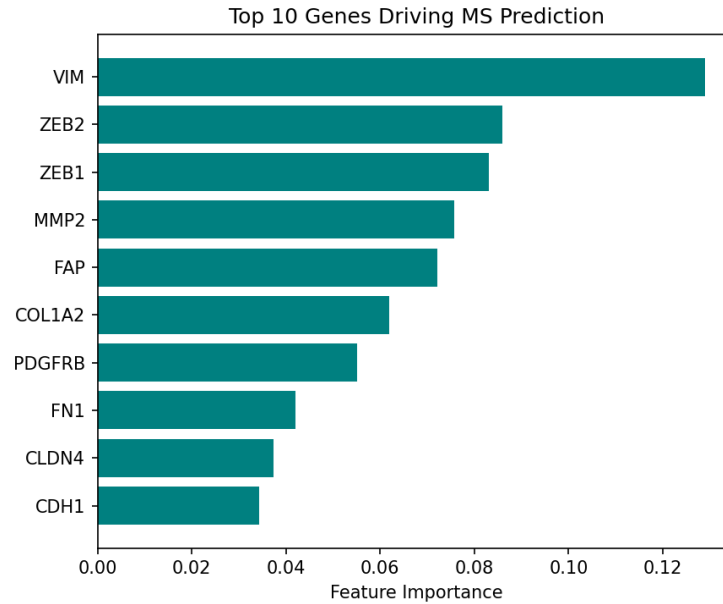


Figure 8: Top 10 genes by Random Forest feature importance. VIM (vimentin), the textbook mesenchymal marker, ranks first, followed by the EMT transcription factors ZEB2 and ZEB1 -- a strong sign the model is learning real biological signal rather than noise.

9.5 Example Prediction Output

Running `python main.py predict --input data/sample_patient.csv` on a real held-out patient (true label: Mesenchymal) produces:

```
{
  "patient_id": "TCGA-CR-7383-01",
  "predicted_subtype": "MS",
  "probabilities": { "MS": 0.913, "Other": 0.087 },
  "top_contributing_genes": ["VIM", "MMP2", "COL1A2", "ZEB2", "ZEB1"]
}
```

The model correctly predicted MS with 91.3% confidence for this patient, matching the ground-truth label.

10. Testing Approach

Unit tests are written with pytest and organized to mirror the five functional modules, in tests/. They can be run with `pytest tests/ -v` after installing dependencies.

Test file	What it covers
test_data_module.py	Merged dataset shape, required columns present, no missing values, binary label correctly derived from the 4-class subtype, unique sample IDs
test_preprocessing_module.py	Schema validation passes on valid data and correctly raises on missing columns, null values, and unexpected labels; train/test split is stratified and non-overlapping
test_training_and_evaluation.py	Trained model only predicts known classes, beats a majority-class baseline, scores above-chance ROC-AUC, and all metrics fall within a valid [0, 1] range
test_prediction_module.py	Prediction output has the expected structure, predicted class is valid, probabilities sum to 1, top genes come from the known gene panel, and invalid input is correctly rejected

Beyond unit tests, the schema validation built into `preprocessing_module.py` and `prediction_module.py` acts as a form of continuous input-testing at runtime -- every dataset load and every prediction request is checked against the expected schema before it reaches the model, rather than being tested only once during development.

11. Challenges Faced

- Sourcing real, legitimately public patient data. The initial instinct was to look for a ready-made "OSCC dataset" download, but real TCGA data is spread across two separate sources that don't share a common identifier format out of the box -- the expression matrix (UCSC Xena) and the subtype labels (a supplementary table in the original 2015 Nature paper) had to be found separately and reconciled.
- Barcode mismatches between sources. The expression matrix uses sample-level barcodes with a tumor/normal suffix (-01 / -11), while the subtype file uses dot-separated patient-level barcodes. Getting the join right required understanding what each suffix meant biologically, not just string-matching.
- Small sample size relative to feature space. With only 279 labelled patients, using the full ~20,000-gene expression matrix would have been a direct invitation to overfit. This pushed the decision toward a curated, literature-grounded gene panel rather than an automatic feature-selection approach, which also happens to make the model more interpretable.
- Class imbalance. MS patients are a minority class (75 of 279). Left unaddressed, a model could score a deceptively high accuracy just by always predicting "Other". Stratified splitting and class-balanced weighting were both needed to get a model that actually learns to distinguish the two classes rather than defaulting to the majority.
- Keeping the pipeline honest. It would have been easy to quietly use the full test set to also pick hyperparameters, inflating the reported performance. Keeping the 5-fold CV (used to sanity-check the model during training) and the held-out test evaluation genuinely separate was something to be deliberate about.

12. Learnings & Key Takeaways

- The curse of dimensionality isn't just a theoretical warning from lecture slides -- with a real 279-sample, 20,000-feature dataset sitting right there, it was very concrete why using all of it without careful feature selection would be a mistake.
- Feature importance from a Random Forest is a genuinely useful sanity check, not just a nice-to-have visualization. Seeing that the top-ranked gene (VIM) is the textbook mesenchymal marker in the literature was reassuring evidence that the model had learned real biology, not an artifact of the data split.
- Class imbalance and stratification matter more in practice than they seem to in theory -- an unstratified split could easily have produced a CV fold with almost no MS patients, making that fold's score meaningless.
- Real-world data wrangling (barcode formats, tumor-vs-normal suffixes, merging two independently-published files) is a much bigger part of an ML project than model training itself. The actual `RandomForestClassifier().fit()` call took under a second; getting to a clean, correctly-merged dataset took considerably longer.
- Building in schema validation and clear error handling from the start, rather than bolting it on afterward, made the whole pipeline much easier to trust and debug.

13. Future Enhancements

- Extend to full 4-class subtype classification (Basal / Mesenchymal / Atypical / Classical) rather than the current binary MS-vs-Other framing.
- Validate on an independent, external cohort (e.g. a GEO expression dataset) to check the model generalizes beyond the TCGA cohort it was trained on.
- Run a proper hyperparameter search (grid search or randomized search) instead of the current fixed configuration.
- Compare against gradient-boosted tree models (XGBoost / LightGBM) as an alternative to Random Forest.
- Replace the heuristic "importance x expression" per-patient gene ranking with a proper SHAP-based explanation for more rigorous per-prediction interpretability.
- Incorporate clinical covariates (age, tumor stage, HPV status) alongside gene expression, since these are known to interact with molecular subtype in the literature.
- Wrap the existing CLI pipeline in a lightweight web interface for non-technical users, while keeping the CLI as the primary, scriptable interface.

14. References

- [1] The Cancer Genome Atlas Network. "Comprehensive genomic characterization of head and neck squamous cell carcinomas." *Nature* 517, 576-582 (2015). <https://doi.org/10.1038/nature14129>
- [2] Walter, V., Yin, X., Wilkerson, M. D., Cabanski, C. R., Zhao, N., Du, Y., et al. "Molecular Subtypes in Head and Neck Cancer Exhibit Distinct Patterns of Chromosomal Gain and Loss of Canonical Cancer Genes." *PLoS ONE* 8(2): e56823 (2013). <https://doi.org/10.1371/journal.pone.0056823>
- [3] UCSC Xena Browser -- TCGA Head and Neck Cancer (HNSC) cohort, gene expression RNAseq (IlluminaHiSeq). <https://xenabrowser.net/datapages/>
- [4] Breiman, L. "Random Forests." *Machine Learning* 45, 5-32 (2001).
- [5] Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12, 2825-2830 (2011). <https://scikit-learn.org/>
- [6] McKinney, W. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference* (2010). (pandas)